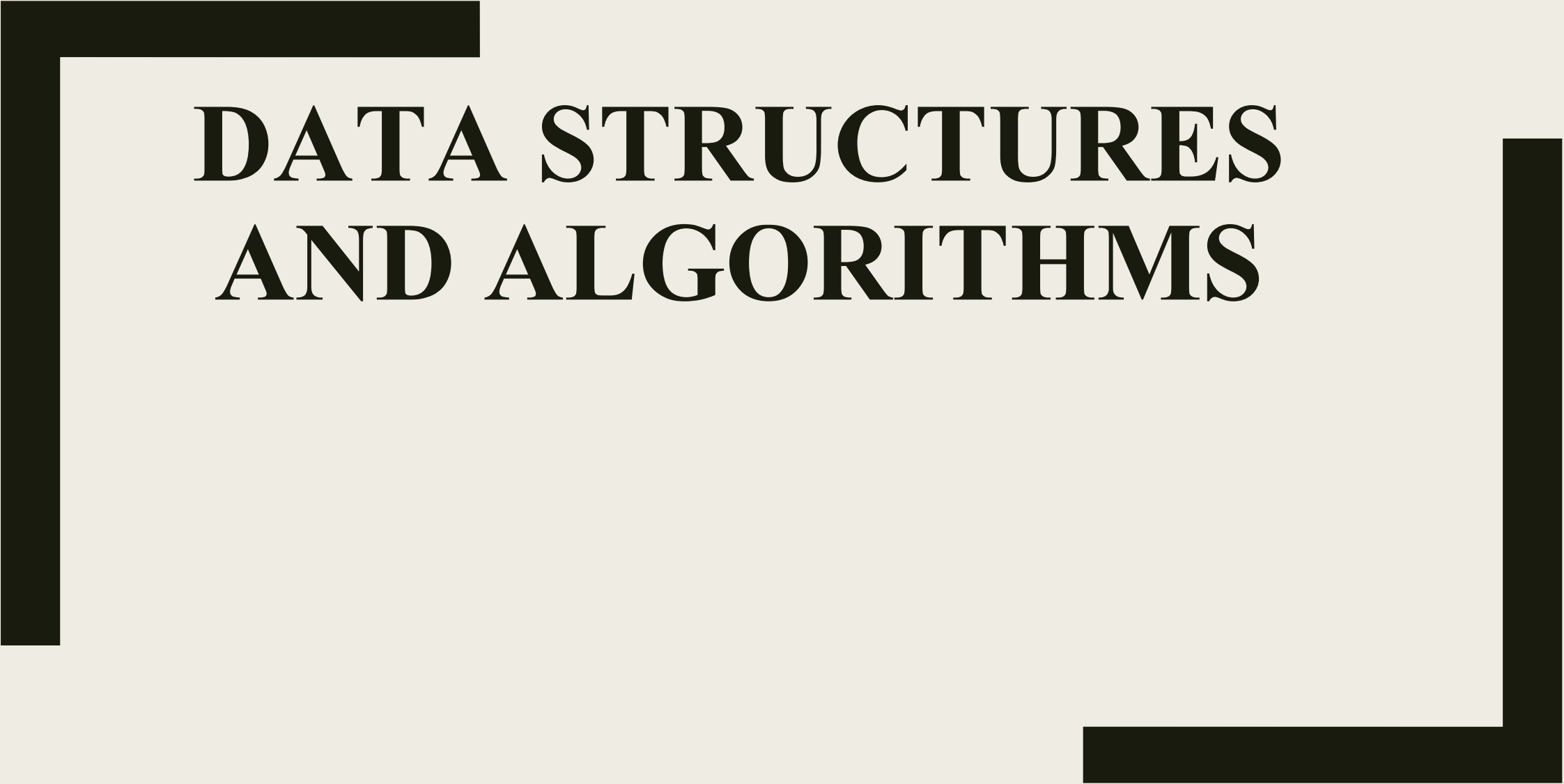


A thick black L-shaped frame is positioned around the text. It starts at the top left, goes right, then down, then right again, forming a large 'C' shape that frames the text.

DATA STRUCTURES AND ALGORITHMS

Overview

- **Introduction to Data Structures**
- **Data Structure Types**
- **What is an Algorithm**
- **Data Structure & Algorithm Importance**

What do you know about

- Data
- Data Structures
- Algorithm

What is Data

- In general, data is any set of characters that is gathered and translated for some purpose, usually analysis.
- In a computer's storage, data is a series of bits (binary digits) that have the value one or zero. Data is processed by the CPU, which uses logical operations to produce new data (output) from source data (input).
- For Example,

Joe, Smith, 1234 Circle, 8404, 8015533144

What is a Data Structure ?

- Data Structures is one of the most fundamental subject in Computer Science & in-depth understanding of this topic is very important especially when you are into development/programming domain where you build efficient software systems & applications.

Definition –

- In computer science, a data structure is a **data organization, management and storage** format that enables **efficient access and modification**.

In Simple Words –

- Data Structure **is a way** in which **data** is **stored** on a computer.

Data Structure

Data structure is a particular way of storing and organizing information in a computer so that it can be retrieved and used most productively.

- Each Data Structure allows data to be stored in specific manner.
- Data Structure allows efficient data search and retrieval.
- Specific Data structures are decided to work for specific problems.
- It allows to manage large amount of data such as large databases.

Why Data Structure

As applications are getting complex and data rich, there are three common problems that applications face now-a-days.

1. **Data Search** – Consider an inventory of 1 million(10^6) items of a store. If the application is to search an item, it has to search an item in 1 million(10^6) items every time slowing down the search. As data grows, search will become slower.
2. **Processor speed** – Processor speed although being very high, falls limited if the data grows to billion records.
3. **Multiple requests** – As thousands of users can search data simultaneously on a web server, even the fast server fails while searching the data.

Importance of DSA

- Data structures helps to store data efficiently and with the help of algorithms we achieve certain task on the given data using lesser resources.
- For instance, consider students in a class
 - *Input records*
 - How?
 - *Manipulate it*
 - *Display it*

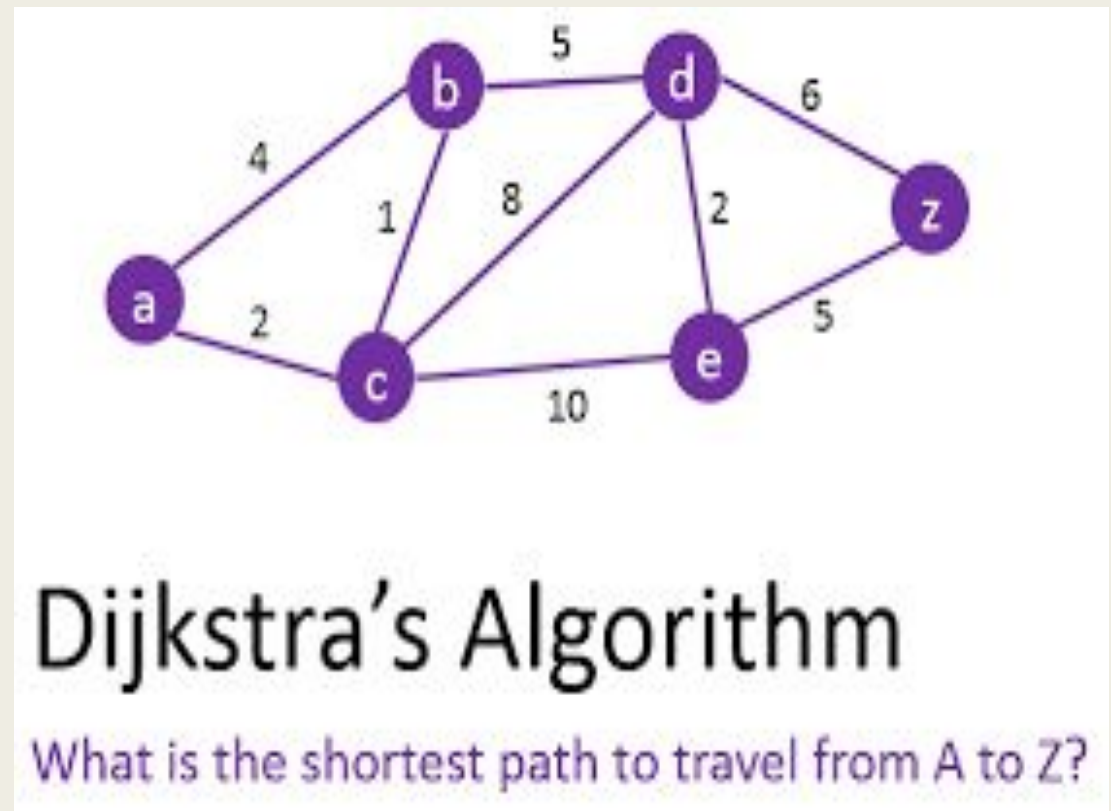
Application of Data Structure

- From the data structure point of view, following are some important operations of algorithms –
- **Search** – Algorithm to search an item in a data structure.
- **Sort** – Algorithm to sort items in a certain order.
- **Insert** – Algorithm to insert item in a data structure.
- **Update** – Algorithm to update an existing item in a data structure.
- **Delete** – Algorithm to delete an existing item from a data structure.

Application of Data Structure

■ The following computer problems can be solved using Data Structures

- *Tower of Hanoi*
- *All pair shortest path by Floyd-Warshall*
- *Shortest path by Dijkstra*
- *Project scheduling*



Static vs. Dynamic Structures

A ***static data structure*** has a fixed size

- This meaning is different from the meaning of the `static` modifier in java
- Arrays are static; once you define the number of elements it can hold, the number doesn't change

A ***dynamic data structure*** grows and shrinks at execution time as required by its contents

- A dynamic data structure is implemented using *links*
- Is *ArrayList* dynamic or not?

Data Types

- Primitive data type
- Non-Primitive data type
- Abstract data Type

Primitive Data Type

- These data types are built-in or predefined data types and can be used directly by the user to declare variables.
- named by a reserved keyword.
- boolean , byte , char , short , int , long , float and double

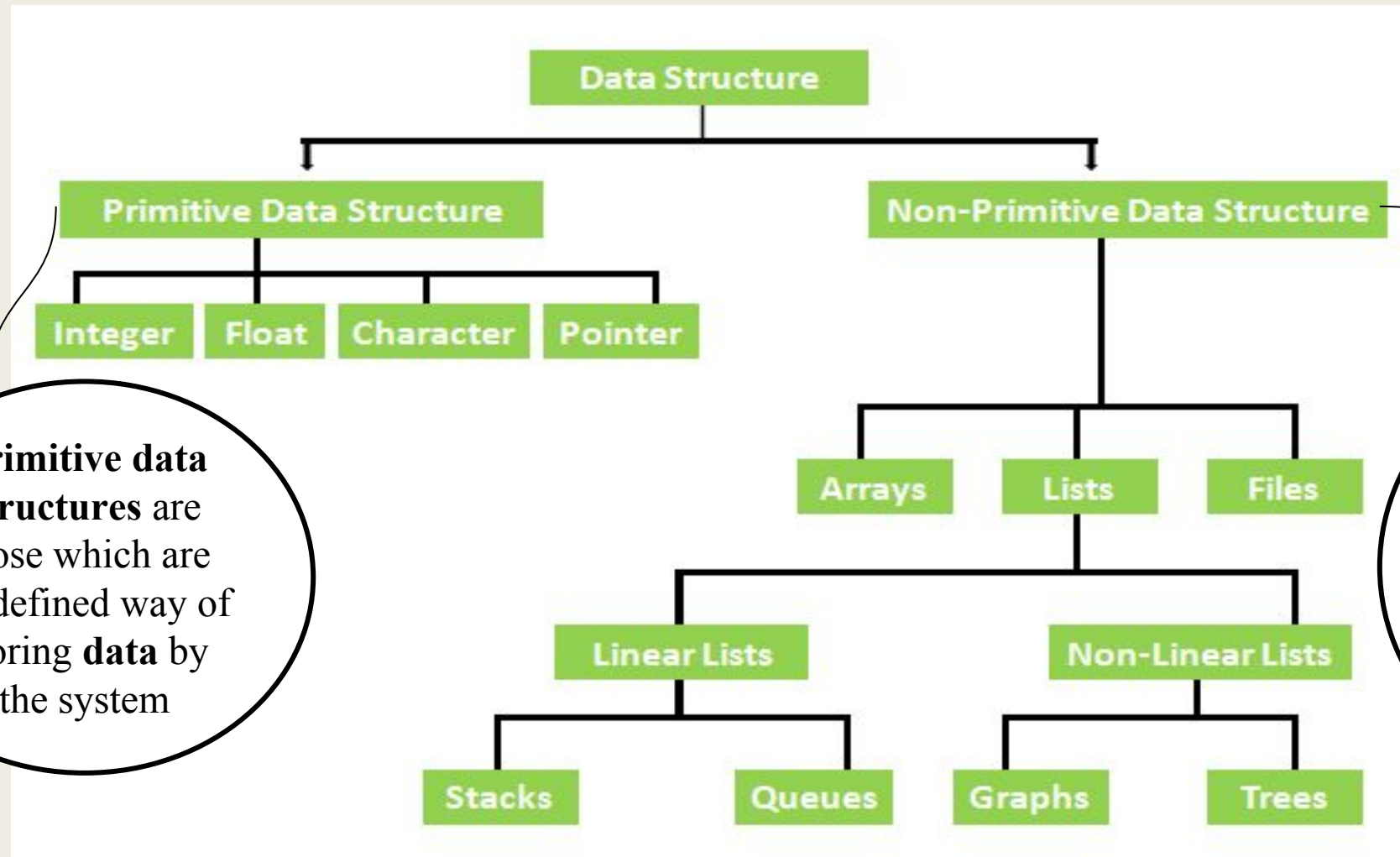
Non-Primitive data type

- Any data type which can be constructed in a program using the programming language's primitive data types and other composite type
- Array, struct, etc.

Abstract data type

- Data type is defined by its behaviour (semantics) from the point of view of a user of the data, specifically in terms of possible values, possible operations on data of this type, and the behaviour of the operations
- Useful tool for specifying the logical properties of a data type
- List, stack, queues, trees, and graphs etc.

Basic Structure of Data Types



Primitive data structures are those which are predefined way of storing **data** by the system

Non-primitive data structures are more complicated **data structures** and are derived from **primitive data structures**.

What is an Algorithm ?

Dictionary Definition –

- A **process** or **set of rules** to be followed in **calculations** or other **problem-solving operations**, especially by a computer.

Formal Definition –

- An **algorithm** is a **finite set of instructions** that are carried in a **specific order** to perform **specific task**.

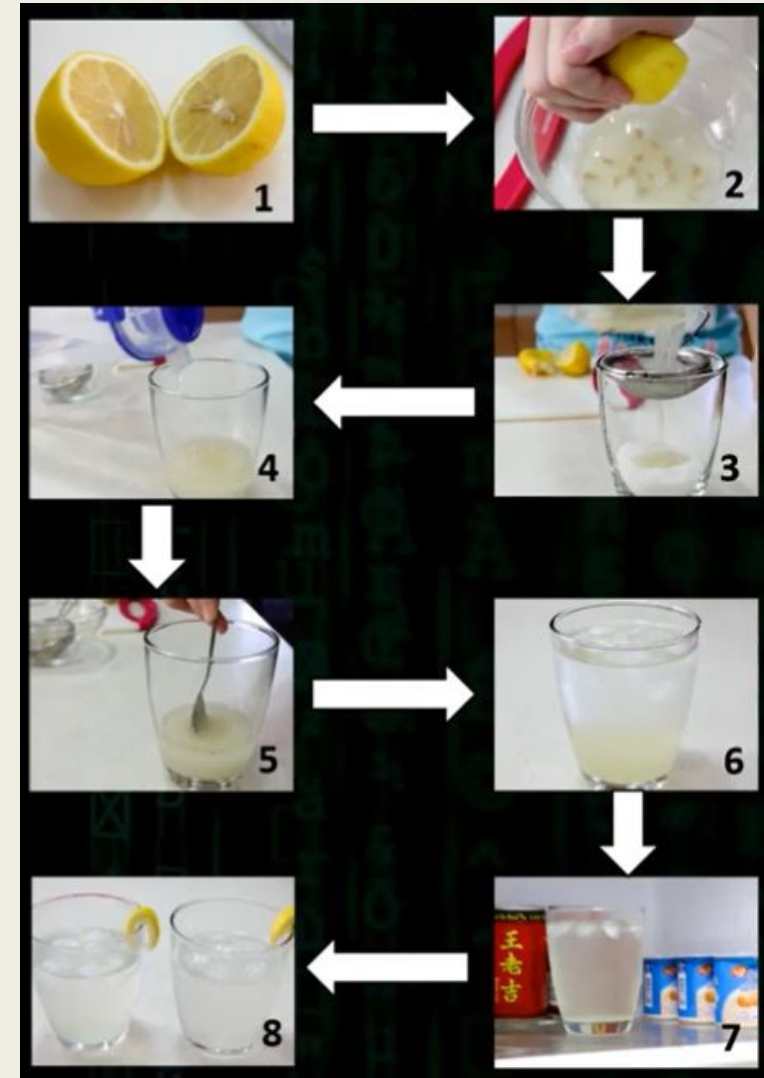
Algorithms typically have the following characteristics –

- **Inputs** : 0 or more input values.
- **Outputs** : 1 or more than 1 output.
- **Unambiguity** : clear and simple instructions.
- **Finiteness** : Limited number of instructions.
- **Effectiveness** : Each instruction has an impact on the overall process.

Real world example of an Algorithm –

Algorithm to make a lemonade –

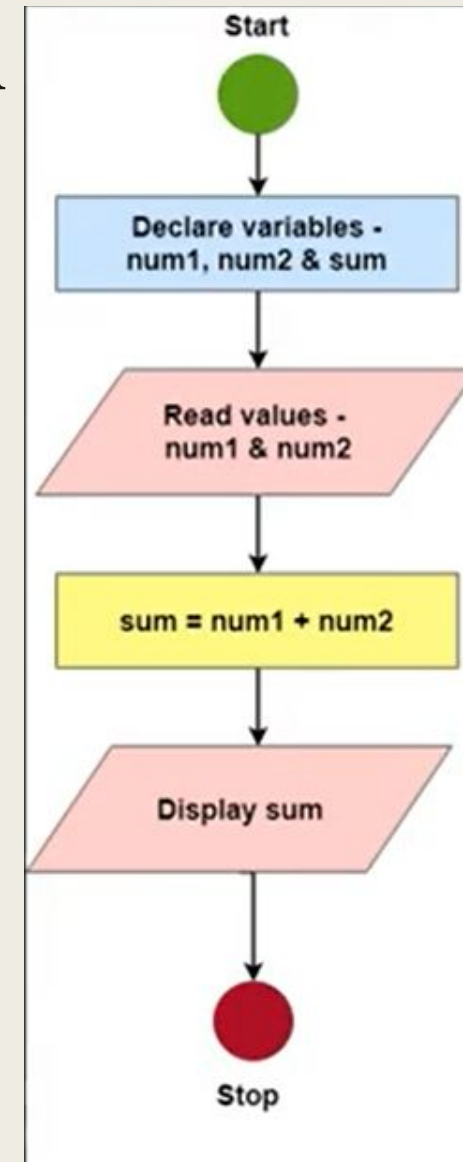
1. Cut your lemon in half.
2. Squeeze all the juice out of it that you can.
3. Pour your juice into a container with 1/4 cup (2 oz) sugar.
4. Add a very small amount of water to your container.
5. Stir your solution until sugar dissolves.
6. Fill up container with water and add ice.
7. Put your lemonade in the fridge for five minutes.
8. Serve and enjoy!



Example of an Algorithm in Programming –

Write an algorithm to add two numbers entered by user. –

- Step 1: Start
- Step 2: Declare variables num1, num2 and sum.
- Step 3: Read values num1 and num2.
- Step 4: Add num1 and num2 and assign the result to sum.($\text{sum} \leftarrow \text{num1} + \text{num2}$)
- Step 5: Display sum
- Step 6: Stop



Practice Example

- Let us now create an algorithm to check whether a number is positive or negative

```
1. Print "Give any number"  
2. Read num  
3. if (num==0) print "You entered 0"  
4. if (num>0) print "You entered a positive number"  
5. if (num<0) print "You entered a negative number"
```

Comparing Algorithms

- Given 2 or more algorithms to solve the same problem, how do we select the best one?
- Some criteria for selecting an algorithm
 - 1) Is it easy to implement, understand, modify?
 - 2) How long does it take to run it to completion? (Time Complexity)
 - 3) How much of computer memory does it use? (Space Complexity)
- Software engineering is primarily concerned with the first criteria
- In this course we are interested in the second and third criteria